



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	Listas Secuenciales.
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Tercero - B
Alumnos participantes:	Camacho Monta Josue Jampier Chalco Tasna Kenneth Mateo Cunalata Mendoza Damian Alexander Silva Camuendo Luis Alexander Tacuri Santillan Monica Sara Tisalema Guashco Darwin Joel
Asignatura:	Estructura de Datos
Docente:	Ing. José Caiza, Mg.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Desarrollar habilidades en el trabajo con listas secuenciales

Específicos:

- Implementar listas secuenciales para gestionar de forma ordenada el catálogo de torres, mapas y oleadas de enemigos ("Cozy").
- Aplicar colas circulares y pilas para controlar el procesamiento por quantum de tiempo de los enemigos y el historial de deshacer/rehacer torres.
- Validar la lógica del simulador evaluando la correcta interacción entre las estructuras de datos y las mecánicas defensivas del juego.

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 8

No presenciales: 0

2.4 Instrucciones

1. Revise la documentación de cómo utilizar la interfaz
2. Revise la documentación de cómo utilizar la clase Stack en:
3. Diseñe la solución al caso de estudio adjunto.
4. Implemente la solución propuesta
5. Elabore un informe del trabajo realizado. Incluya en él el diagrama de clases que incluya las clases fundamentales de su propuesta de solución.
6. Suba el informe a la plataforma.
7. Nota: solo un integrante de cada grupo deberá realizar la entrega

2.5 Listado de equipos, materiales y recursos

- Inteligencia artificial
- TAC
- PC de escritorio o portátil con JDK instalado e IDE de su elección.

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

☒ Plataformas educativas



- ☒ Simuladores y laboratorios virtuales
- ☐ Aplicaciones educativas
- ☐ Recursos audiovisuales
- ☐ Gamificación
- ☒ Inteligencia Artificial
- Otros (Especifique): _____

2.6 Actividades por desarrollar

Comparar la implementación de pilas y colas utilizando las clases e interfaces que provee Java.

2.7 Resultados obtenidos

Resultados obtenidos de laS Listas Secuenciales.

1. Ejercicio

Diseñar e implementar un simulador simplificado de un juego de defensa de torres, en el que el jugador coloca torres para defenderse de oleadas de enemigos (obligatoriamente llamados Cozy). **El sistema debe emplear:**

- Colas para gestionar el movimiento de los enemigos a lo largo de una ruta fija y para gestionar el orden de las oleadas.
- Pilas para implementar el historial de colocación y mejora de torres, permitiendo deshacer y rehacer dichas acciones.

El enemigo pierde vida (PV) al recibir daño (DD) de las torres en rango; si un Cozy llega al final de la ruta, el jugador pierde una vida. El juego finaliza cuando el jugador se queda sin vida (derrota) o cuando se superan todas las oleadas (victoria).

2. Solución propuesta

Se organiza el proyecto en tres paquetes, cada uno con una responsabilidad clara:

Paquete	Clase	Responsabilidad
modelo	Coordenada	Representa una posición (fila, columna) del mapa; calcula distancia Manhattan.
modelo	Cozy	Enemigo del juego: PV, velocidad, posición en la ruta, recompensa.
modelo	Torre (abstracta)	Atributos y comportamiento común de toda torre: daño, alcance, nivel, ataque.
modelo	TorreArquero / TorreCanon	Especializaciones de Torre con estadísticas y progresión de mejora propias.
modelo	CatalogoEnemigos	Fábrica de instancias Cozy, escalando PV según la dificultad de la oleada.
modelo	CatalogoTorres	Fábrica de instancias Torre a partir del tipo elegido por el jugador.
modelo	Oleada	Agrupar el conjunto de Cozy liberados en una oleada, con dificultad creciente.
estructuras	Mapa	Administra la ruta fija y el arreglo de torres colocadas.
estructuras	ColaEnemigos	Cola circular que actualiza a cada Cozy en cada quantum de tiempo.
estructuras	GestorOleadas	Cola FIFO que libera las oleadas pendientes en orden.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026



estructuras	ComandoTorre (interfaz)	Contrato del patrón Command: ejecutar() / deshacer().
estructuras	ComandoColocarTorre / ComandoMejorarTorre	Comandos concretos para colocar y mejorar torres.
estructuras	HistorialComandos	Doble pila (deshacer/rehacer) sobre comandos de torre.
gestor	GestorJuego	Orquesta vida, puntaje, oleada actual y coordina las demás estructuras.
—	Main	Punto de entrada; menú por consola.

3. ¿Qué son las estructuras utilizadas y por qué se aplican aquí?

Cola (Queue): estructura FIFO donde el primer elemento en entrar es el primero en salir. En este proyecto se usa de dos formas:

- **Cola circular (ColaEnemigos):** cada Cozy vivo que no llega al final se vuelve a encolar al final de la misma cola, simulando su avance continuo por la ruta durante cada quantum de tiempo.
- **Cola FIFO simple (GestorOleadas):** las oleadas se liberan en el mismo orden en que fueron agregadas.

Pila (Stack/Deque): estructura LIFO donde el último elemento en entrar es el primero en salir. Se usa en HistorialComandos mediante dos pilas (pilaDeshacer y pilaRehacer) para implementar el patrón Command, permitiendo revertir y reaplicar acciones sobre torres sin que el historial conozca los detalles internos de cada

4. Relación entre requisitos y código

Requisito	Aplicación en el código
Cola circular para el movimiento del enemigo	ColaEnemigos.actualizarQuantum(): fija el tamaño antes del bucle y re-encola a cada Cozy vivo que sigue en ruta.
Cola para el orden de las oleadas	GestorOleadas (Deque<Oleada> con ArrayDeque).
Pila para historial de colocación/mejora	HistorialComandos (Deque<ComandoTorre> con ArrayDeque, usado como pila mediante push()/pop()).
Deshacer colocación/mejora	ComandoColocarTorre.deshacer() y ComandoMejorarTorre.deshacer(), invocados desde HistorialComandos.deshacer().
Rehacer una acción deshecha	HistorialComandos.rehacer(), que vuelve a ejecutar el comando y lo regresa a la pila de deshacer.
Retiro de un enemigo cuando sus PV llegan a 0	Cozy.estaVivo() (pv > 0) verificado en ColaEnemigos.actualizarQuantum(), que no re-encola al Cozy muerto.
Enemigo obligatorio "Cozy"	Clase modelo.Cozy.
Herencia	TorreArquero extends Torre, TorreCanon extends Torre.
Polimorfismo	Torre.mejorar() es abstracto y cada subclase define su propio incremento; atacar() se ejecuta igual para cualquier tipo de torre gracias a la referencia Torre.



Encapsulamiento	Atributos private/protected en Mapa, Cozy, Torre, HistorialComandos, con acceso mediante métodos públicos.
Listas secuenciales de catálogos	CatalogoTorres.TIPOS[] y CatalogoEnemigos.TIPOS[].

5. Paso 1: Importar las estructuras de colas y pilas de Java

```
import java.util.ArrayDeque;  
import java.util.Deque;
```

Deque/ArrayDeque se usa en las tres estructuras del proyecto: como cola en ColaEnemigos y GestorOleadas, y como pila LIFO en HistorialComandos. Las tres se apoyan en ArrayDeque en lugar de LinkedList o Stack por el mejor rendimiento medido en la sección 20 “Comparación teórica pilas/colas”.

6. Paso 2: Crear la cola circular de enemigos

```
public class ColaEnemigos {  
    private final Deque<Cozy> colaCircular;  
  
    public void actualizarQuantum(Mapa mapa, GestorJuego estado) {  
        int cantidadAProcesar = colaCircular.size();  
        for (int i = 0; i < cantidadAProcesar; i++) {  
            Cozy c = colaCircular.poll();  
            // ... ataque de torres, muerte, avance o fuga ...  
            if (sigueVivoYEnRuta) {  
                colaCircular.add(c); // vuelve al final: comportamiento circular  
            }  
        }  
    }  
}
```

El tamaño se fija antes del bucle para que cada Cozy se procese una sola vez por quantum, aun cuando se vuelva a encolar dentro del mismo ciclo. Esto es lo que le da el carácter "circular" a la cola.

7. Paso 3: Crear la pila de historial mediante el patrón Command

```
public interface ComandoTorre {  
    void ejecutar();  
    void deshacer();  
}  
  
public class HistorialComandos {  
    private final Deque<ComandoTorre> pilaDeshacer;  
    private final Deque<ComandoTorre> pilaRehacer;  
  
    public void registrar(ComandoTorre c) {  
        pilaDeshacer.push(c);  
        pilaRehacer.clear(); // una acción nueva invalida el rehacer pendiente  
    }  
}
```

Encapsular cada acción en un objeto ComandoTorre permite que HistorialComandos deshaga/rehaga sin conocer si se trató de una colocación o una mejora.



8. Paso 4: Aplicar herencia entre torres

```
public abstract class Torre {  
    protected int dano, alcance, nivel;  
    public abstract void mejorar();  
}  
  
public class TorreArquero extends Torre {  
    public TorreArquero(Coordenada pos) { super(10, 3, 50, pos); }  
    @Override public void mejorar() { nivel++; dano += 5; }  
}  
  
public class TorreCanon extends Torre {  
    public TorreCanon(Coordenada pos) { super(25, 2, 90, pos); }  
    @Override public void mejorar() { nivel++; dano += 10; }  
}
```

Ambas subclases reutilizan estaEnRango(), atacar() y restaurar() definidos en Torre, y solo redefinen su progresión de mejora.

9. Paso 5: Aplicar polimorfismo en el ataque y la mejora

```
for (Torre t : torresActivas) {  
    if (c.estaVivo() && t.estaEnRango(posActual)) {  
        t.atacar(c); // el mismo método sirve para TorreArquero o TorreCanon  
    }  
}
```

La referencia es de tipo Torre, pero en tiempo de ejecución se invoca el mejorar() correspondiente a TorreArquero o TorreCanon.

10. Paso 6: Registrar, deshacer y rehacer acciones sobre torres

```
public void colocarTorreJugador(Torre t) {  
    ComandoColocarTorre comando = new ComandoColocarTorre(t, mapa);  
    comando.ejecutar();  
    historial.registrar(comando);  
}  
  
public void deshacerUltimaAccion() {  
    if (historial.puedeDeshacer()) historial.deshacer();  
}
```

ComandoColocarTorre guarda si la colocación fue exitosa (exitosa), de modo que deshacer una colocación fallida (casilla ocupada) no retire ninguna torre. ComandoMejorarTorre guarda una fotografía (nivelAnterior, danoAnterior, alcanceAnterior) antes de mejorar, para poder restaurarla exactamente.

11. Paso 7: Instanciar torres y enemigos mediante fábricas

```
Torre torre = CatalogoTorres.crear(indiceTipo, new Coordenada(fila, columna));  
Cozy enemigo = CatalogoEnemigos.crear(indiceTipo, factorDificultad);
```

CatalogoTorres y CatalogoEnemigos centralizan la creación de objetos, evitando que Main u Oleada conozcan los constructores concretos de cada subtipo.



12. Tipos de datos utilizados

Tipo	Categoría	Uso	Descripción
pv, pvMax, indiceRuta, velocidad, recompensa, dano, alcance, nivel, costo, fila, columna	int	Primitivo	Contadores y estadísticas numéricas enteras.
— (estaVivo(), isExitosa())	boolean	Primitivo	Resultados lógicos de validación.
tipo (Cozy), nombre (tipos de catálogo)	String	Referencia / No primitivo	Nombre identificador de enemigos y torres.
Cozy[], Torre[], Coordenada[]	Arreglos de objetos	Referencia / No primitivo	Contenido de una oleada, torres activas del mapa, ruta del mapa.
Deque<Cozy>, Deque<Oleada>	Colección	Referencia / No primitivo	ColaEnemigos y GestorOleadas (ambas con ArrayDeque).
Deque<ComandoTorre>	Colección	Referencia / No primitivo	Pilas de deshacer/rehacer en HistorialComandos (ArrayDeque).

13. Diagrama de clases

```
classDiagram
class Coordinada {
    -int fila
    -int columna
    +distanciaA(Coordinada) int
}
class Cozy {
    -String tipo
    -int pv
    -int pvMax
    -int indiceRuta
    -int velocidad
    -int recompensa
    +recibirDano(int) void
    +estaVivo() boolean
    +avanzar() void
    +llegoAlFinal(int) boolean
}
class Torre {
    <<abstract>>
    #int dano
    #int alcance
    #int nivel
    #Coordinada posicion
    +estaEnRango(Coordinada) boolean
    +atacar(Cozy) void
    +mejorar()* void
    +restaurar(int,int,int) void
}
class TorreArquero
class TorreCanon
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026



```
class Mapa {
    -Coordenada[] ruta
    -Torre[] torres
    +colocarTorre(Torre) boolean
    +quitarTorre(Torre) void
    +buscarTorreEn(Coordenada) Torre
}
class ColaEnemigos {
    -Deque~Cozy~ colaCircular
    +encolar(Cozy) void
    +actualizarQuantum(Mapa, GestorJuego) void
}
class GestorOleadas {
    -Deque~Oleada~ oleadas
    +agregarOleada(Oleada) void
    +siguienteOleada() Oleada
}
class ComandoTorre {
    <<interface>>
    +ejecutar() void
    +deshacer() void
}
class ComandoColocarTorre
class ComandoMejorarTorre
class HistorialComandos {
    -Deque~ComandoTorre~ pilaDeshacer
    -Deque~ComandoTorre~ pilaRehacer
    +registrar(ComandoTorre) void
    +deshacer() void
    +rehacer() void
}
class GestorJuego {
    -int vidaJugador
    -int puntuacion
    -int oleadaActual
    +procesarQuantum() void
    +colocarTorreJugador(Torre) void
    +deshacerUltimaAccion() void
}

Torre <|-- TorreArquero
Torre <|-- TorreCanon
ComandoTorre <|.. ComandoColocarTorre
ComandoTorre <|.. ComandoMejorarTorre
Mapa o-- Torre
Mapa o-- Coordenada
ColaEnemigos o-- Cozy
GestorOleadas o-- Oleada
HistorialComandos o-- ComandoTorre
GestorJuego --> ColaEnemigos
GestorJuego --> GestorOleadas
GestorJuego --> HistorialComandos
GestorJuego --> Mapa
```




14. Especificación de la estructura

Estado:

- **vidaJugador, puntuacion, oleadaActual:** enteros que reflejan el progreso de la partida.
- **colaEnemigos:** cola circular con los Cozy actualmente en la ruta.
- **gestorOleadas:** cola FIFO con las oleadas pendientes.
- **historial:** doble pila de comandos de torre.
- **mapa:** ruta fija y arreglo de torres colocadas.

Operaciones:

Método	Resultado
procesarQuantum()	Carga la siguiente oleada si la ruta está vacía y avanza un ciclo de simulación.
colocarTorreJugador(Torre)	Crea, ejecuta y registra un ComandoColocarTorre.
mejorarTorreJugador(Torre)	Crea, ejecuta y registra un ComandoMejorarTorre.
deshacerUltimaAccion()	Deshace la última acción registrada, si existe.
rehacerUltimaAccion()	Rehace la última acción deshecha, si existe.
restarVida(int)	Descuenta vida al jugador por cada fuga de un Cozy.
sumarPuntaje(int)	Suma la recompensa de un Cozy eliminado.
juegoTerminado()	Indica si $\text{vidaJugador} \leq 0$.
haGanado()	Indica si ya no hay oleadas pendientes ni enemigos en ruta.

Invariantes:

1. $0 \leq \text{vidaJugador}$.
2. $0 \leq \text{cantidadTorres} \leq \text{capacidadTorres en Mapa}$.
3. La ruta es fija y no cambia durante la partida.
4. Una torre nunca puede colocarse sobre una casilla ya ocupada por otra torre.
5. Un Cozy con $\text{pv} \leq 0$ se retira de ColaEnemigos y no vuelve a encolarse.
6. Cada Cozy se procesa exactamente una vez por quantum, aunque se re-encole en el mismo ciclo.
7. Toda acción nueva registrada en HistorialComandos invalida la pila de rehacer vigente (`pilaRehacer.clear()`).
8. `deshacer()` solo puede ejecutarse si `pilaDeshacer` no está vacía; `rehacer()` solo si `pilaRehacer` no está vacía.
9. Deshacer una colocación fallida (casilla ocupada o mapa lleno) no retira ninguna torre, porque exitosa = false.
10. Las oleadas se liberan en el mismo orden en que fueron agregadas (política FIFO de GestorOleadas).

15. Pseudocódigo

INICIO

crear Mapa, ColaEnemigos, GestorOleadas, HistorialComandos
instanciar GestorJuego
precargar oleadas (dificultad creciente)



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026



REPETIR

mostrar estado (vida, oleada, puntaje, mapa)

mostrar menú

leer opción

SEGÚN opción HACER

1: leer tipo y posición -> colocarTorreJugador()

2: leer posición de torre existente -> mejorarTorreJugador()

3: deshacerUltimaAccion()

4: rehacerUltimaAccion()

5: procesarQuantum()

6: finalizar

SI juegoTerminado() ENTONCES

mostrar "Derrota"; finalizar

SINO SI haGanado() ENTONCES

mostrar "Victoria"; finalizar

FIN SI

HASTA QUE opción = 6 O el juego haya terminado

FIN

16. Prueba de escritorio

Caso	Operación realizada	Estado previo	Resultado observado
Cola vacía	colaEnemigos.actualizarQuantum() con colaCircular vacía	Sin oleada cargada	cantidadAProcesar = 0; el bucle no se ejecuta y no ocurre ningún efecto.
Una oleada	procesarQuantum() cuando gestorOleadas tiene una oleada pendiente y la ruta está vacía	oleadaActual = 0	Se extrae la oleada con siguienteOleada(), se encola cada Cozy y oleadaActual se actualiza.
Deshacer sin haber colocado nada	deshacerUltimaAccion()	pilaDeshacer vacía	puedeDeshacer() retorna false; no se llama a historial.deshacer(); no hay cambios ni excepción.
Rehacer sin haber deshecho	rehacerUltimaAccion()	pilaRehacer vacía	puedeRehacer() retorna false; no se llama a historial.rehacer(); no hay cambios ni excepción.
Colocar torre en casilla ocupada	colocarTorreJugador(t2) con t2.getPosicion() igual a una torre ya existente	buscarTorreEn (pos) distinto de null	mapa.colocarTorre(t2) retorna false; comando.exitosa = false; queda registrado en el historial pero deshacer() posterior no quitará ninguna torre.
Enemigo llega al final	c.avanzar() hace que indiceRuta >= longitudRuta - 1	Cozy vivo, cerca del final de la ruta	llegoAlFinal() retorna true; estado.restarVida(DANO_POR_FUGA); el Cozy no se re-encola (se retira de la ruta).



Resultados adicionales del APE 02 Listas Secuenciales

17. Estado del sistema (atributos de todas las clases)

Clase	Atributos
Coordenada	fila: int, columna: int
Cozy	tipo: String, pv: int, pvMax: int, indiceRuta: int, velocidad: int, recompensa: int
Torre (abstracta)	dano: int, alcance: int, costo: int, nivel: int, posicion: Coordenada
TorreArquero	(hereda de Torre, sin atributos propios adicionales)
TorreCanon	(hereda de Torre, sin atributos propios adicionales)
CatalogoEnemigos	TIPOS: TipoEnemigo[] (estático); TipoEnemigo: nombre: String, pvBase: int, velocidad: int, recompensa: int
CatalogoTorres	TIPOS: TipoTorre[] (estático); TipoTorre: nombre: String, costo: int, danoBase: int, alcanceBase: int
Oleada	numero: int, enemigos: Cozy[]
Mapa	ruta: Coordenada[], fuenteEnergia: Coordenada, torres: Torre[], cantidadTorres: int
ColaEnemigos	colaCircular: Deque<Cozy>, DANO_POR_FUGA: int (constante)
GestorOleadas	oleadas: Deque<Oleada>
ComandoTorre	(interfaz, sin atributos)
ComandoColocarTorre	torre: Torre, mapa: Mapa, exitosa: boolean
ComandoMejorarTorre	torre: Torre, nivelAnterior: int, danoAnterior: int, alcanceAnterior: int
HistorialComandos	pilaDeshacer: Deque<ComandoTorre>, pilaRehacer: Deque<ComandoTorre>
GestorJuego	vidaJugador: int, puntuacion: int, oleadaActual: int, colaEnemigos: ColaEnemigos, gestorOleadas: GestorOleadas, historial: HistorialComandos, mapa: Mapa

18. Tabla de operaciones del TDA

Estructura / Clase	Operación	Descripción
ColaEnemigos	encolar(Cozy)	Agrega un enemigo al final de la cola circular.
ColaEnemigos	actualizarQuantum(Mapa, GestorJuego)	Procesa un ciclo: ataque de torres, muerte, avance, fuga o re-encolado.
ColaEnemigos	estaVacía() / tamaño()	Consultan el estado de la cola.
GestorOleadas	agregarOleada(Oleada)	Encola una nueva oleada.
GestorOleadas	siguienteOleada()	Extrae (FIFO) la siguiente oleada pendiente.
GestorOleadas	hayOleadasPendientes()	Consulta si quedan oleadas por liberar.
HistorialComandos	registrar(ComandoTorre)	Apila el comando en pilaDeshacer y limpia pilaRehacer.
HistorialComandos	deshacer()	Extrae el tope de pilaDeshacer, lo deshace y lo mueve a pilaRehacer.
HistorialComandos	rehacer()	Extrae el tope de pilaRehacer, lo ejecuta y lo mueve a pilaDeshacer.
HistorialComandos	puedeDeshacer() / puedeRehacer()	Consultan si las pilas respectivas tienen elementos.
Mapa	colocarTorre(Torre)	Agrega una torre si hay espacio y la casilla está libre.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026



Mapa	quitarTorre(Torre)	Retira una torre y compacta el arreglo.
Mapa	buscarTorreEn(Coordenada)	Ubica una torre por su posición.
Torre	estaEnRango(Coordenada)	Evalúa si un enemigo está dentro del alcance.

19. Invariantes numeradas

1. La ruta es fija y no cambia durante la partida (Mapa.ruta es final y se define una sola vez en el constructor).
2. Una torre nunca puede colocarse sobre una casilla ya ocupada por otra torre (Mapa.colocarTorre() rechaza la operación si buscarTorreEn(pos) != null).
3. $0 \leq \text{cantidadTorres} \leq \text{torres.length}$ en Mapa.
4. $0 < pv \leq pvMax$ mientras el Cozy esté vivo; $pv = 0$ implica que ya fue retirado de la ruta.
5. Un Cozy se retira de ColaEnemigos si $pv \leq 0$ (muerte) o si $\text{indiceRuta} \geq \text{longitudRuta} - 1$ (fuga); en ambos casos no vuelve a encolarse.
6. Cada Cozy se procesa como máximo una vez por quantum, aun si se re-encola dentro del mismo ciclo (cantidadAProcesar se fija antes del bucle).
7. Toda llamada a HistorialComandos.registrar() limpia pilaRehacer, de modo que nunca coexisten una rama de "rehacer" antigua con una acción nueva.
8. HistorialComandos.deshacer() y rehacer() solo se ejecutan si la pila correspondiente no está vacía (verificado por puedeDeshacer()/puedeRehacer()) antes de invocarlos desde GestorJuego).
9. Una colocación de torre fallida (exitosa = false) no debe eliminarse al deshacer, aunque quede registrada en el historial.
10. Las oleadas se procesan en orden estrictamente FIFO: la oleada n nunca se libera antes que la oleada n-1.
11. vidaJugador nunca es negativo ($\text{Math.max}(0, \text{vidaJugador} - \text{cantidad})$).
12. El juego declara derrota (juegoTerminado()) antes de evaluar victoria (haGanado()), evitando que una partida perdida se reporte como ganada.

20. Comparación teórica: pilas vs. colas

Aspecto	Pila (LIFO) — HistorialComandos	Cola (FIFO / circular) — ColaEnemigos, GestorOleadas
Orden de acceso	Último en entrar, primero en salir.	Primero en entrar, primero en salir.
Operación de inserción	push() — $O(1)$	add() / offer() — $O(1)$
Operación de extracción	pop() — $O(1)$	poll() — $O(1)$
Consulta del próximo elemento	peek() (tope) — $O(1)$	peek() (frente) — $O(1)$
Uso en el proyecto	Deshacer/rehacer de comandos de torre.	Movimiento continuo de enemigos (circular) y orden de liberación de oleadas (FIFO).
Justificación de la elección	El deshacer/rehacer requiere revertir siempre la acción más reciente, comportamiento natural de una pila.	El avance de enemigos y el orden de las oleadas dependen de quién llegó primero, comportamiento natural de una cola.



21. Resultados empíricos

Para esta comparación se ejecutaron 200 000 operaciones sobre cada estructura, repitiendo la prueba tres veces para no quedarse con un número que pudiera ser casualidad de una sola corrida, ya que la JVM tarda un poco en calentar y eso puede meter ruido en la primera pasada.

Estructura	Operación	Tiempo (ms)
Stack	push/pop	31–36
ArrayDeque (pila)	push/pop	19–33
LinkedList (cola)	offer/poll	23–26
ArrayDeque (cola)	offer/poll	11–15

En la cola la diferencia es clara y se repite en las tres corridas: ArrayDeque le gana a LinkedList casi por el doble. En la pila la ventaja de ArrayDeque sobre Stack también aparece, aunque con más variación entre corridas cuando no se calienta la JVM con varias vueltas antes de medir.

Esto pasa porque Stack en realidad es una reliquia de las primeras versiones de Java: por debajo extiende de Vector, y todos sus métodos están marcados como synchronized. Eso significa que cada push() o pop() toma y suelta un candado (lock) pensado para cuando varios hilos usan la misma pila al mismo tiempo, algo que este juego, al ser de un solo hilo, jamás necesita. Es trabajo de más que no compra nada.

Con la cola pasa algo parecido, pero por otro motivo: LinkedList guarda cada elemento en un nodo separado, enlazado al siguiente. Cada vez que entra un dato se crea un nodo nuevo en memoria, y cada vez que sale, se libera; eso implica más trabajo para el recolector de basura y los datos terminan dispersos en la memoria en vez de estar juntos. ArrayDeque, en cambio, usa un solo arreglo circular por debajo, así que no anda creando y destruyendo nodos, y todo queda más cerca en memoria (mejor uso de caché).

Con esto, la recomendación para el proyecto es usar ArrayDeque en los tres casos, tal como ya quedó implementado en HistorialComandos, ColaEnemigos y GestorOleadas; estas dos últimas originalmente usaban LinkedList y ya fueron actualizadas a ArrayDeque.

2.8 Habilidades blandas empleadas en la práctica

- ☐ Liderazgo
- ☒ Trabajo en equipo
- ☐ Comunicación asertiva
- ☐ La empatía
- ☒ Pensamiento crítico
- ☐ Flexibilidad
- ☐ La resolución de conflictos
- ☐ Adaptabilidad
- ☒ Responsabilidad

2.9 Conclusiones

- Se demostró la utilidad de las listas secuenciales para acceder y organizar eficientemente los elementos estáticos de la simulación.
- La combinación de colas circulares y pilas resolvió con precisión la actualización de salud en tiempo real y la gestión de comandos.
- El simulador confirmó que la integración adecuada de estructuras de datos garantiza un flujo de juego estable y coherente.



2.10 Recomendaciones

- Optimizar el tamaño de las listas secuenciales para evitar un consumo innecesario de memoria al ampliar el contenido del juego.
- Validar los límites en pilas y colas para prevenir errores de desbordamiento al procesar múltiples oleadas o deshacer acciones.
- Mantener un código modular que facilite añadir nuevas mecánicas o tipos de estructuras en futuras versiones del sistema.

2.11 Referencias bibliográficas

[1] Baeldung, "Reduce Object Header Size and Save Memory in Java 25," Baeldung, may 2026. Accedido el 30 de agosto de 2026. [En línea]. Disponible: <https://www.baeldung.com/java-object-header-reduced-size-save-memory>

[2] "The Cost of Garbage Collection for State Machine Replication," arXiv preprint, arXiv:2405.11182, 2024. Accedido el 30 de agosto de 2026. [En línea]. Disponible: <https://arxiv.org/pdf/2405.11182> .

2.12 Anexos

EVIDENCIA DE LAS EJECUCIONES DE LOS CÓDIGOS.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026



```
Vida: 20 | Oleada: 0 | Puntaje: 0
E . . . .
. . . .
. . . . F
1) Colocar torre
2) Mejorar torre
3) Deshacer
4) Rehacer
5) Avanzar quantum
6) Salir
Elige una opcion: 1
0) Arquero - Costo: 50 | Dano: 10 | Alcance: 3
1) Canon - Costo: 90 | Dano: 25 | Alcance: 2
Elige el tipo de torre: 0
Fila: 2
Columna: 2
Torre colocada.
Vida: 20 | Oleada: 0 | Puntaje: 0
E . . . .
. . . .
. . A . F
1) Colocar torre
2) Mejorar torre
3) Deshacer
4) Rehacer
5) Avanzar quantum
6) Salir
Elige una opcion: 1
0) Arquero - Costo: 50 | Dano: 10 | Alcance: 3
1) Canon - Costo: 90 | Dano: 25 | Alcance: 2
Elige el tipo de torre: 1
Fila: 1
Columna: 1
Torre colocada.
Vida: 20 | Oleada: 0 | Puntaje: 0
E . . . .
. C . . .
. . A . F
1) Colocar torre
2) Mejorar torre
3) Deshacer
4) Rehacer
5) Avanzar quantum
6) Salir
Elige una opcion: 5
Vida: 20 | Oleada: 1 | Puntaje: 40
E . . . .
. C . . .
. . A . F
1) Colocar torre
2) Mejorar torre
3) Deshacer
4) Rehacer
5) Avanzar quantum
6) Salir
Elige una opcion: 5
Vida: 20 | Oleada: 2 | Puntaje: 40
E . . . .
. C . . .
. . A . F
1) Colocar torre
2) Mejorar torre
3) Deshacer
4) Rehacer
5) Avanzar quantum
6) Salir
```

Figura 1. Ejecución del código en C++.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026



```
E . . . .
. C . . .
. . A . F
1) Colocar torre
2) Mejorar torre
3) Deshacer
4) Rehacer
5) Avanzar quantum
6) Salir
Elige una opcion: 5
Vida: 20 | Oleada: 5 | Puntaje: 252
E . . . .
. C . . .
. . A . F
1) Colocar torre
2) Mejorar torre
3) Deshacer
4) Rehacer
5) Avanzar quantum
6) Salir
Elige una opcion: 5
Vida: 20 | Oleada: 5 | Puntaje: 312
E . . . .
. C . . .
. . A . F
1) Colocar torre
2) Mejorar torre
3) Deshacer
4) Rehacer
5) Avanzar quantum
6) Salir
Elige una opcion: 5
Vida: 20 | Oleada: 6 | Puntaje: 312
E . . . .
. C . . .
. . A . F
1) Colocar torre
2) Mejorar torre
3) Deshacer
4) Rehacer
5) Avanzar quantum
6) Salir
Elige una opcion: 5
Vida: 20 | Oleada: 6 | Puntaje: 366
E . . . .
. C . . .
. . A . F
1) Colocar torre
2) Mejorar torre
3) Deshacer
4) Rehacer
5) Avanzar quantum
6) Salir
Elige una opcion: 5
Vida: 20 | Oleada: 6 | Puntaje: 366
E . . . .
. C . . .
. . A . F
1) Colocar torre
2) Mejorar torre
3) Deshacer
4) Rehacer
5) Avanzar quantum
6) Salir
Elige una opcion: 5
Vida: 20 | Oleada: 6 | Puntaje: 426
E . . . .
. C . . .
```

Figura 2. Ejecución del código en C++.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026



```
E . . . .
. C . . .
. . A . F
1) Colocar torre
2) Mejorar torre
3) Deshacer
4) Rehacer
5) Avanzar quantum
6) Salir
Elige una opcion: 5
Vida: 20 | Oleada: 7 | Puntaje: 560
E . . . .
. C . . .
. . A . F
1) Colocar torre
2) Mejorar torre
3) Deshacer
4) Rehacer
5) Avanzar quantum
6) Salir
Elige una opcion: 5
Vida: 20 | Oleada: 8 | Puntaje: 560
E . . . .
. C . . .
. . A . F
1) Colocar torre
2) Mejorar torre
3) Deshacer
4) Rehacer
5) Avanzar quantum
6) Salir
Elige una opcion: 5
Vida: 20 | Oleada: 8 | Puntaje: 622
E . . . .
. C . . .
. . A . F
1) Colocar torre
2) Mejorar torre
3) Deshacer
4) Rehacer
5) Avanzar quantum
6) Salir
Elige una opcion: 5
Vida: 20 | Oleada: 8 | Puntaje: 622
E . . . .
. C . . .
. . A . F
1) Colocar torre
2) Mejorar torre
3) Deshacer
4) Rehacer
5) Avanzar quantum
6) Salir
Elige una opcion: 5
Vida: 20 | Oleada: 8 | Puntaje: 702
E . . . .
```

Figura 3. Ejecución del código en C++.



```
E . . . .  
. C . . .  
. . A . F  
1) Colocar torre  
2) Mejorar torre  
3) Deshacer  
4) Rehacer  
5) Avanzar quantum  
6) Salir  
Elige una opcion: 5  
Vida: 20 | Oleada: 9 | Puntaje: 854  
E . . . .  
. C . . .  
. . A . F  
1) Colocar torre  
2) Mejorar torre  
3) Deshacer  
4) Rehacer  
5) Avanzar quantum  
6) Salir  
Elige una opcion: 5  
Vida: 20 | Oleada: 10 | Puntaje: 854  
E . . . .  
. C . . .  
. . A . F  
1) Colocar torre  
2) Mejorar torre  
3) Deshacer  
4) Rehacer  
5) Avanzar quantum  
6) Salir  
Elige una opcion: 5  
Vida: 20 | Oleada: 10 | Puntaje: 886  
E . . . .  
. C . . .  
. . A . F  
1) Colocar torre  
2) Mejorar torre  
3) Deshacer  
4) Rehacer  
5) Avanzar quantum  
6) Salir  
Elige una opcion: 5  
Vida: 20 | Oleada: 10 | Puntaje: 926  
E . . . .  
. C . . .  
. . A . F  
1) Colocar torre  
2) Mejorar torre  
3) Deshacer  
4) Rehacer  
5) Avanzar quantum  
6) Salir  
Elige una opcion: 5  
Vida: 20 | Oleada: 10 | Puntaje: 926  
E . . . .  
. C . . .  
. . A . F  
1) Colocar torre  
2) Mejorar torre  
3) Deshacer  
4) Rehacer  
5) Avanzar quantum  
6) Salir  
Elige una opcion: 5  
¡Victoria!  
Puntaje final: 1026 | Oleada alcanzada: 10  
PS C:\Users\Jampier_M\Documents\APE_02\Estructura_Datos_APE2>
```

Figura 4. Ejecución del código en C++.



```
Vida: 20 | Oleada: 0 | Puntaje: 0
E . . . .
. . . . .
. . . . F
1) Colocar torre
2) Mejorar torre
3) Deshacer
4) Rehacer
5) Avanzar quantum
6) Salir
Elige una opcion: 1
0) Arquero - Costo: 50 | Dano: 10 | Alcance: 3
1) Canon - Costo: 90 | Dano: 25 | Alcance: 2
Elige el tipo de torre: 1
Fila: 2
Columna: 2
Torre colocada.
Vida: 20 | Oleada: 0 | Puntaje: 0
E . . . .
. . . . .
. . C . F
1) Colocar torre
2) Mejorar torre
3) Deshacer
4) Rehacer
5) Avanzar quantum
6) Salir
Elige una opcion: 1
0) Arquero - Costo: 50 | Dano: 10 | Alcance: 3
1) Canon - Costo: 90 | Dano: 25 | Alcance: 2
Elige el tipo de torre: 0
Fila: 1
Columna: 2
Torre colocada.
Vida: 20 | Oleada: 0 | Puntaje: 0
E . . . .
. . A . .
. . C . F
1) Colocar torre
```

Figura 5. Ejecución del código en Java.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026



```
1) Colocar torre
2) Mejorar torre
3) Deshacer
4) Rehacer
5) Avanzar quantum
6) Salir
Elige una opcion: 3
Vida: 20 | Oleada: 0 | Puntaje: 0
E . . . .
. . . .
. . C . F
1) Colocar torre
2) Mejorar torre
3) Deshacer
4) Rehacer
5) Avanzar quantum
6) Salir
Elige una opcion: 4
Vida: 20 | Oleada: 0 | Puntaje: 0
E . . . .
. . A . .
. . C . F
1) Colocar torre
2) Mejorar torre
3) Deshacer
4) Rehacer
5) Avanzar quantum
6) Salir
Elige una opcion: 5
Vida: 20 | Oleada: 1 | Puntaje: 0
E . . . .
. . A . .
. . C . F
E . . . .
. . A . .
. . C . F
1) Colocar torre
2) Mejorar torre
3) Deshacer
4) Rehacer
5) Avanzar quantum
6) Salir
Elige una opcion: 5
Vida: 20 | Oleada: 10 | Puntaje: 926
E . . . .
. . A . .
. . C . F
1) Colocar torre
2) Mejorar torre
3) Deshacer
4) Rehacer
5) Avanzar quantum
6) Salir
Elige una opcion: 5
Vida: 20 | Oleada: 10 | Puntaje: 926
E . . . .
. . A . .
. . C . F
1) Colocar torre
2) Mejorar torre
3) Deshacer
4) Rehacer
5) Avanzar quantum
6) Salir
Elige una opcion: 5
¡Victoria!
Puntaje final: 1026 | Oleada alcanzada: 10
```

Figura 6. Ejecución del código en Java.





UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026

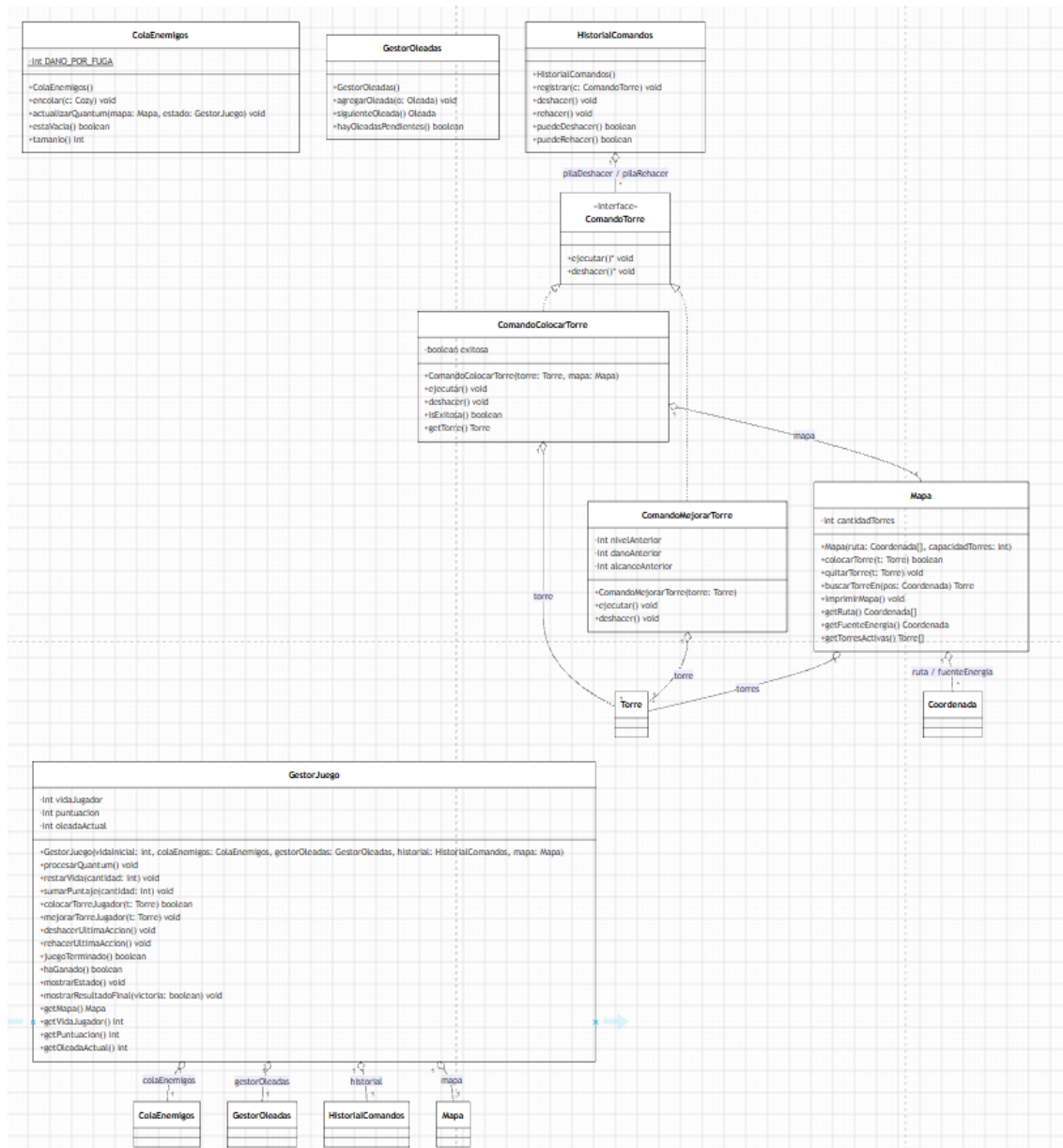


Figura 9. Diagrama UML Paquete Gestor.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026

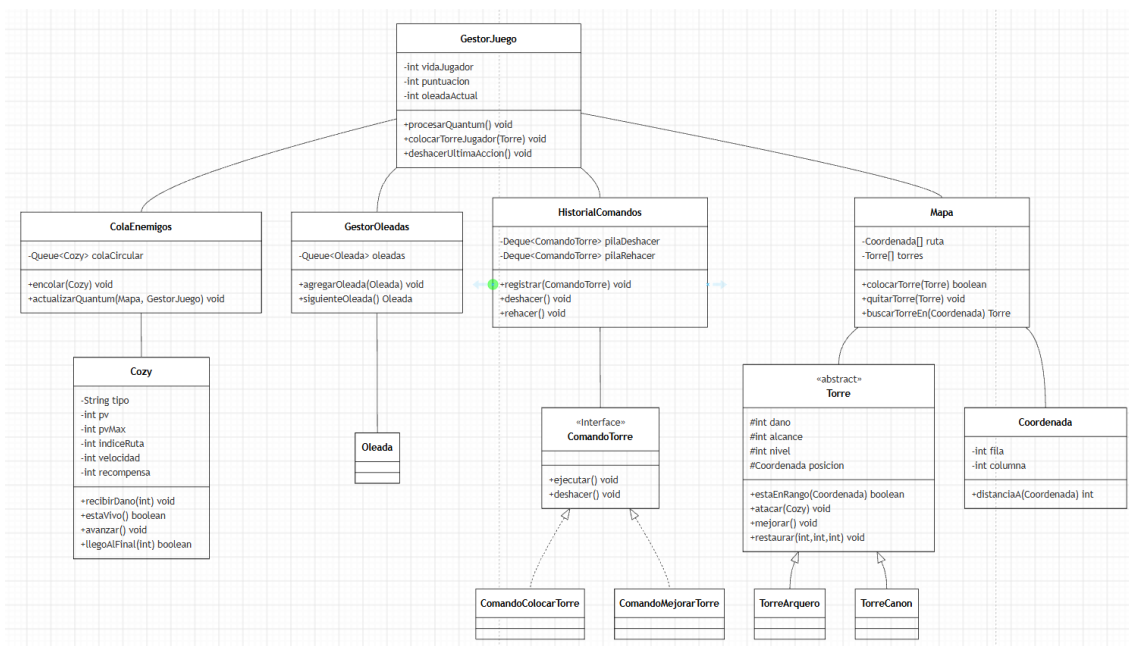


Figura 10. Diagrama UML General Juego.

En el siguiente enlace al repositorio, se encuentran disponibles para su visualización los códigos desarrollados Java. Asimismo, se adjuntan las evidencias del diagrama UML correspondiente y el presente informe en formatos .pdf.

Enlace repositorio: https://github.com/Damienq7w/Estructura_Datos_APE2.git